Somali Mobile Money Data Dictionary

MySQL table and column reference

Database somali_wallet_source | Version 1 | 24 October 2026

Use this reference to understand each source table, choose the correct join keys, and interpret the payment records before building your extraction and analysis pipeline.

The project models one fictional provider handling person-to-person transfers in USD. All people and transactions are generated. Every recorded failure or uncertainty reason comes from the simulated payment system.

Table directory

Table	Meaning of one row	Default rows
customers	One row per synthetic customer.	5,000
wallets_opening	One row per wallet at the opening snapshot.	5,000
wallets_closing	One row per wallet at the closing snapshot.	5,000
payment_requests	One row per logical payment.	100,000
payment_attempts	One row per attempt at a logical payment.	100,766
event_deliveries	One row per received event copy, not per payment or unique event.	205,362
ledger_entries	One wallet movement belonging to a successful attempt.	168,106
system_incidents	One simulated infrastructure incident.	15
reason_codes	One row per recognised system reason code.	7
dataset_metadata	One row per named metadata document; the loader currently creates one.	1 after loading

Coverage 10 MySQL tables and 64 columns. Counts describe the supplied default dataset, not live database measurements.

Examples are illustrative. They explain field formats and do not claim that a particular example ID has those values in the supplied dataset.

How to read the dictionary

A table's grain describes what one row represents. Choosing the wrong grain is a common cause of incorrect payment counts.

Term	Meaning
Payment	The customer's intention to send money. One payment_id can have several attempts.
Attempt	One execution try. A confirmed retryable failure may lead to another attempt.
Event	One state change for an attempt, identified by event_id.
Delivery	One received copy of an event. Duplicate deliveries share event_id but have distinct delivery_id values.
PK and FK	PK means primary key, the row's unique identifier. FK means foreign key, a reference to a row in another table.
Required and Nullable	Required means SQL NOT NULL or an implicit primary-key requirement. Nullable allows SQL NULL. Empty CSV fields become NULL during loading.

Money and time

Amounts ending in cents use exact integers: 2500 means USD 25.00. signed_amount_cents can be negative. No fees, cash deposits, withdrawals, or currency conversion are modelled.

DATETIME(3) stores milliseconds. The loader stores UTC values; MySQL DATETIME does not itself attach a timezone. Convert UTC to UTC+03:00 for the Somali business calendar. Event time, feed arrival time, and extraction time are different clocks.

Main table relationships

Parent	Child reference	Relationship
customers.customer_id	wallets_opening.customer_id	One customer to many wallets allowed; one generated.
wallets_opening.wallet_id	wallets_closing.wallet_id	At most one closing row per opening wallet.
wallets_opening.wallet_id	payment_requests sender and recipient IDs	Each payment has two wallet roles.
payment_requests.payment_id	payment_attempts.payment_id	One payment to one or more attempts in the dataset.
payment_attempts.attempt_id	event_deliveries.attempt_id	One attempt to zero or more received event copies.
payment_attempts.attempt_id	ledger_entries.attempt_id	Two postings per successful attempt; none otherwise.
reason_codes.reason_code	event_deliveries.reason_code	Optional reason lookup.
system_incidents.incident_id	event_deliveries.incident_id	Optional incident context.

Customers and dataset settings

Customers

customers

Identifies the people who own the simulated wallets. Use it to attach a customer name and home city to wallet activity.

Grain One row per synthetic customer.

Source data/source/customers.csv | Default rows 5,000

Column	Type and rules	Meaning and example
customer_id	VARCHAR(24) PK	Unique identifier for a customer. Join this to wallets_opening.customer_id. Names are not identifiers. **Example** CUS_000001
full_name	VARCHAR(150) Required	Invented Somali name. Different customers may share the same name; do not join or deduplicate on this field. **Example** Ayaan Xasan Cali
home_city	VARCHAR(50) Required	The customer's assigned home city. The generator uses an assumed city distribution, not measured population shares. **Example** Muqdisho
registered_at	DATETIME(3) Required	When the fictional customer registered. This precedes the payment observation period. **Example** 2025-06-15 09:00:00.000 UTC

The generator creates one wallet per customer. The SQL foreign key itself permits multiple wallets to reference the same customer.

Dataset metadata

dataset_metadata

Records the generator settings and observation window used for this dataset. This helps explain assumptions and reproduce the same simulation.

Grain One row per named metadata document; the loader currently creates one.

Source Created by load_mysql.py from data/generation_config.json | Default rows 1 after loading

Column	Type and rules	Meaning and example
metadata_key	VARCHAR(50) PK	Unique name of the settings document stored in this row. **Example** generation_config
metadata_json	JSON Required	JSON object containing the seed, volumes, dates, probabilities, and project assumptions. The object's fields are explained later in this document. **Example** {"seed":42,"currency":"USD",...}

This is an actual MySQL table. The generator's schema.json is a separate file describing CSV headers and row counts; it is not a MySQL table.

Opening wallet balances

wallets_opening

Provides the wallet register and each wallet's balance at the start of the observation period. It is the parent table for wallet references throughout the schema.

Grain One row per wallet at the opening snapshot.

Source data/source/wallets_opening.csv | Default rows 5,000

Column	Type and rules	Meaning and example
wallet_id	VARCHAR(24) PK	Unique wallet identifier. Payments reference this key separately for the sender and the recipient. **Example** WAL_000001
customer_id	VARCHAR(24) Required / FK	Owner of the wallet. References customers.customer_id. **Example** CUS_000001
currency	CHAR(3) Required	Currency of the wallet. The generated project uses USD only. **Example** USD
status	VARCHAR(12) Required	Static wallet status in this simulation: ACTIVE or INACTIVE. An inactive recipient causes a rejection. **Example** ACTIVE
opening_balance_cents	BIGINT Required	Money in the wallet before the simulated transfers begin, stored as an exact integer number of cents. **Example** 15000 means USD 150.00
snapshot_at	DATETIME(3) Required	Time at which this starting balance applies. The default start is 1 July 2026 at midnight local time. **Example** 2026-06-30 21:00:00.000 UTC

Opening balances are starting values, not the available balances before every later payment. Wallet status stays fixed in this dataset.

Using the opening balance

A wallet may send and receive money many times after the opening snapshot. To calculate a later ledger balance, add the wallet's signed ledger entries up to the chosen time. The available balance may be smaller if money is held for another attempt.

Closing wallet balances

wallets_closing

Shows balances after processing all system actions before the observation cutoff. Use it to reconcile opening balances with posted ledger movements.

Grain One row per wallet at the closing snapshot.

Source data/source/wallets_closing.csv | Default rows 5,000

Column	Type and rules	Meaning and example
wallet_id	VARCHAR(24) PK / FK	The wallet being measured. This is both the row's primary key and a reference to wallets_opening.wallet_id. **Example** WAL_000001
currency	CHAR(3) Required	Currency of the snapshot. Always USD in the generated data. **Example** USD
snapshot_at	DATETIME(3) Required	Exclusive observation cutoff. Events at or after this instant are outside the simulation's observed results. **Example** 2026-09-28 21:00:00.000 UTC
ledger_balance_cents	BIGINT Required	Total wallet balance after posted incoming and outgoing transfers. Includes money currently held for unresolved processing. **Example** 10000 means USD 100.00
held_balance_cents	BIGINT Required	Part of the balance reserved for processing and not available for a new transfer. A hold is not another debit. **Example** 2000 means USD 20.00 reserved
available_balance_cents	BIGINT Required	Money available for new transfers: ledger balance minus held balance. **Example** 8000 means USD 80.00 available

The schema enforces ledger balance = held balance + available balance, with all three nonnegative. Use this final snapshot only at the final cutoff; it was not available at the start of the period.

A balance example

Balance	Cents	Meaning
Ledger balance	10000	USD 100.00 is recorded in the wallet.
Held balance	2000	USD 20.00 is reserved for unresolved processing.
Available balance	8000	USD 80.00 remains available for new transfers.

Payment requests

payment_requests

Records the customer's logical intention to send an amount to another wallet. A request remains the same payment even when a confirmed failure is retried.

Grain One row per logical payment.

Source data/source/payment_requests.csv | Default rows 100,000

Column	Type and rules	Meaning and example
payment_id	VARCHAR(24) PK	Unique identifier for the logical payment. All its attempts, state events, and ledger entries refer to this ID. **Example** PAY_0000001
requested_at	DATETIME(3) Required	Time of the original request. This stays unchanged when an attempt is retried. **Example** 2026-07-02 08:15:00.000 UTC
sender_wallet_id	VARCHAR(24) Required / FK	Wallet that would send the money. References wallets_opening.wallet_id. **Example** WAL_000001
recipient_wallet_id	VARCHAR(24) Required / FK	Wallet intended to receive the money. References wallets_opening.wallet_id and must differ from the sender. **Example** WAL_000002
amount_cents	BIGINT Required	Requested amount in cents. It is positive, but a request may exceed a limit or available balance and therefore fail. **Example** 2500 means USD 25.00
currency	CHAR(3) Required	Currency of the requested transfer. Always USD in this dataset. **Example** USD
payment_type	VARCHAR(10) Required	Type of payment. P2P means person to person; that is the only type generated. **Example** P2P
channel	VARCHAR(10) Required	How the request was initiated. USSD represents a phone menu; APP represents a mobile application. **Example** USSD or APP
origin_city	VARCHAR(50) Required	Assigned from the sender's home city. It is a grouping attribute, not a measured GPS or network location. **Example** Baydhabo

There is no final status column here. Derive outcomes from attempts and events, checking the ledger where appropriate.

For payment-level metrics, count distinct payment_id. Summing every attempt's requested amount can repeat the same payment.

Payment attempts

payment_attempts

Records each execution attempt at a payment, including the balance available when that attempt started. This separates individual failures from the eventual result of the payment.

Grain One row per attempt at a logical payment.

Source data/source/payment_attempts.csv | Default rows 100,766

Column	Type and rules	Meaning and example
attempt_id	VARCHAR(32) PK	Unique identifier for one attempt. The generated format combines the payment ID with an attempt suffix. **Example** PAY_0000001_A1
payment_id	VARCHAR(24) Required / FK	The logical payment being attempted. References payment_requests.payment_id. **Example** PAY_0000001
attempt_number	INT Required	Sequence number within the payment, starting at 1. The generator allows up to three attempts; SQL enforces a positive number and uniqueness within the payment. **Example** 1, 2, or 3
started_at	DATETIME(3) Required	Time this particular attempt began. A retry has a later start time than the previous attempt. **Example** 2026-07-02 08:15:00.000 UTC
available_balance_cents_at_start	BIGINT Required	Sender's ledger balance minus existing holds at attempt start. Compare it with the requested amount when checking an insufficient-funds rejection. **Example** 1000 means USD 10.00 available

The pair payment_id and attempt_number is unique. A new attempt is created only after a confirmed retryable failure, never merely because a response timed out.

A retry example

The following example has one payment and two attempts. The first failed; the second succeeded. It therefore contributes one failed attempt, one successful attempt, and one successful payment.

Payment	Attempt	Result
PAY_0000001	PAY_0000001_A1	FAILED with SERVICE_UNAVAILABLE
PAY_0000001	PAY_0000001_A2	SUCCEEDED with no failure reason

The attempt table does not contain a final status. Reconstruct that status from the latest business event for the attempt, then cross-check ledger evidence when events are missing.

Payment event deliveries

event_deliveries

One row is one event copy received by the analytics log feed. Duplicates and late arrivals are preserved for cleaning and state reconstruction.

Source data/source/event_deliveries/*.csv | Default rows 205,362

Column	Type and rules	Meaning and example
source_row_id	BIGINT PK Auto ID	MySQL-generated extraction cursor for this single-writer, append-only exercise. It is absent from the CSV files. **Example** 1
event_id	VARCHAR(24) Required	Identifier of the underlying system event. Duplicate copies share this ID, so this column is deliberately not unique. **Example** EVT_000000001
payment_id	VARCHAR(24) Required / FK	Logical payment associated with the event. References payment_requests.payment_id. **Example** PAY_0000001
attempt_id	VARCHAR(32) Required / FK	Attempt whose state changed. References payment_attempts.attempt_id. **Example** PAY_0000001_A1
event_at	DATETIME(3) Required	When the payment system produced the state change. Use this timestamp to order the business history. **Example** 2026-07-02 08:15:10.000 UTC
status	VARCHAR(12) Required	State reported by this event: REQUESTED, PENDING, SUCCEEDED, or FAILED. It may not be the latest state. **Example** PENDING
reason_code	VARCHAR(48) Nullable / FK	Optional code referencing reason_codes.reason_code. RESPONSE_TIMEOUT means uncertainty, not confirmed failure. **Example** RESPONSE_TIMEOUT
reason_source	VARCHAR(12) Nullable	SYSTEM whenever a reason code exists; otherwise NULL. No customer-entered reasons are recorded. **Example** SYSTEM or NULL
system_component	VARCHAR(40) Required	Simulated component emitting the event or reason. Ordinary events without a reason use PAYMENT_ENGINE. **Example** PAYMENT_GATEWAY
incident_id	VARCHAR(24) Nullable / FK	Optional link to a simulated incident whose scope matched the attempt at its start. References system_incidents.incident_id. **Example** INC_0001 or NULL
delivery_id	VARCHAR(24) Required Unique	Unique identifier for the received copy. Two copies of one event have different delivery IDs. **Example** DEL_000000001
ingested_at	DATETIME(3) Required	When this copy reached the simulated analytics feed. May be much later than event_at; it is not the later S3 extraction time. **Example** 2026-07-02 09:20:00.000 UTC

Wallet ledger entries

ledger_entries

Records money movements for successful transfers. Each success creates one outgoing entry for the sender and one equal incoming entry for the recipient.

Grain One wallet movement belonging to a successful attempt.

Source data/source/ledger_entries.csv | Default rows 168,106

Column	Type and rules	Meaning and example
entry_id	VARCHAR(24) PK	Unique identifier for one posted movement. A successful transfer has two different entry IDs. **Example** LED_000000001
payment_id	VARCHAR(24) Required / FK	Logical payment responsible for the movement. References payment_requests.payment_id. **Example** PAY_0000001
attempt_id	VARCHAR(32) Required / FK	Successful attempt responsible for posting. References payment_attempts.attempt_id. **Example** PAY_0000001_A1
wallet_id	VARCHAR(24) Required / FK	Wallet whose balance changes. References wallets_opening.wallet_id. **Example** WAL_000001
posted_at	DATETIME(3) Required	Time the system confirmed success and recorded this movement. Both sides of the transfer have the same posting time. **Example** 2026-07-02 08:15:03.000 UTC
signed_amount_cents	BIGINT Required	Negative for money leaving this wallet; positive for money arriving. The two entries for one transfer sum to zero. **Example** -2500 outgoing or +2500 incoming
currency	CHAR(3) Required	Currency of the posting. Always USD in the generated data. **Example** USD
entry_type	VARCHAR(20) Required	Direction of this movement. TRANSFER_OUT requires a negative amount; TRANSFER_IN requires a positive amount. **Example** TRANSFER_OUT or TRANSFER_IN

Confirmed failures and unresolved attempts do not post ledger movements. A successful event missing from the analytics feed may still have ledger evidence.

For payment volume, sum one side of the ledger or unique successful payments. Summing absolute amounts on both sides doubles the volume.

A successful transfer of USD 25

Wallet role	Signed cents	Effect
Sender	-2500	Reduces the sender’s ledger balance by USD 25.00.
Recipient	2500	Increases the recipient’s ledger balance by USD 25.00.

Opening balance plus signed movements equals closing ledger balance. Holds reduce availability without creating extra postings.

System incidents

system_incidents

Describes simulated periods of payment-switch degradation detected by a system monitor. Match the time window, city, and channel when studying the incident's scope.

Grain One simulated infrastructure incident.

Source data/source/system_incidents.csv | Default rows 15

Column	Type and rules	Meaning and example
incident_id	VARCHAR(24) PK	Unique identifier for the incident. Referenced optionally by event_deliveries.incident_id. **Example** INC_0001
started_at	DATETIME(3) Required	Start of the incident window. The matching rule includes this instant. **Example** 2026-07-03 09:00:00.000 UTC
ended_at	DATETIME(3) Required	End of the incident window. The matching rule excludes this instant: start <= attempt time < end. **Example** 2026-07-03 11:00:00.000 UTC
affected_city	VARCHAR(50) Required	City included in the incident's scope. ALL means all simulated cities. **Example** Muqdisho or ALL
affected_channel	VARCHAR(10) Required	Channel included in the scope. The generator uses USSD or ALL; ALL covers both USSD and APP. **Example** USSD or ALL
incident_type	VARCHAR(60) Required	Technical incident category. One category is generated in this version. **Example** SIMULATED_SWITCH_DEGRADATION
recorded_by	VARCHAR(24) Required	Source that recorded the incident. Always SYSTEM_MONITOR in this project. **Example** SYSTEM_MONITOR

An incident increases selected simulated failure and timeout probabilities; it does not make every payment fail. An insufficient-funds rejection during an incident still has insufficient funds as its recorded reason.

Interpreting an incident link

The generator checks the attempt's start time against the incident window and its city and channel scope. Some events may carry incident_id because the attempt fell inside that scope, even if its reason was insufficient funds or a limit rejection. Treat the link as context, then inspect the recorded reason.

System reason definitions

reason_codes

Provides the lookup dictionary for the codes stored on payment events. It distinguishes definitive failures from an uncertain response and identifies which failures may be retried.

Grain One row per recognised system reason code.

Source data/source/reason_codes.csv | Default rows 7

Column	Type and rules	Meaning and example
reason_code	VARCHAR(48) PK	Unique machine-readable label. Referenced by event_deliveries.reason_code. **Example** INSUFFICIENT_FUNDS
reason_class	VARCHAR(16) Required	FAILURE means a definitive rejection or cancellation. UNCERTAINTY means that the final result is not yet known. **Example** FAILURE or UNCERTAINTY
system_component	VARCHAR(40) Required	The simulated component associated with this reason. It supplies context for technical investigation. **Example** BALANCE_ENGINE
description	VARCHAR(255) Required	Human-readable explanation of the code. This is predefined system documentation, not a customer complaint. **Example** Available balance is below the requested amount
retry_after_confirmed_failure	VARCHAR(3) Required	YES permits a retry after definitive failure; NO does not. YES does not guarantee a retry or authorize retrying a pending attempt. **Example** YES or NO

The generator records the first applicable rejection in its check order. A recorded reason is not an exhaustive list of every condition that could block the payment.

Payment event status values

Status	Meaning	Analytical treatment
REQUESTED	The system received the attempt.	An initial state. Still unresolved if no later result is known.
PENDING	Processing has no definitive result yet.	Track age and later confirmations; do not count as a confirmed failure.
SUCCEEDED	The transfer completed and posted.	Count once at the intended payment or attempt grain.
FAILED	The system confirmed rejection or cancellation.	Count as a failed attempt and inspect its failure code.

Failure and uncertainty code reference

These are project-defined codes. They are not claims about a real provider's API. All are emitted by simulated system components.

Reason code	Class and component	Meaning	Retry
INSUFFICIENT_FUNDS	FAILURE BALANCE_ENGINE	Available balance is smaller than the requested amount.	NO
RECIPIENT_INACTIVE	FAILURE ACCOUNT_VALIDATOR	The receiving wallet is inactive.	NO
TRANSFER_LIMIT_EXCEEDED	FAILURE LIMIT_ENGINE	The request exceeds the simulated per-transfer limit.	NO
SERVICE_UNAVAILABLE	FAILURE PAYMENT_SWITCH	The switch definitively rejected the attempt before posting.	YES
PROCESSING_ERROR	FAILURE PAYMENT_ENGINE	Processing definitively failed without a posting.	YES
PROCESSING_EXPIRED	FAILURE PAYMENT_ENGINE	The system confirmed cancellation after uncertainty and released the hold.	YES
RESPONSE_TIMEOUT	UNCERTAINTY PAYMENT_GATEWAY	No final response yet. A later event may confirm success or failure.	NO

A timeout can later succeed

Step	System event	What the analyst knows
1	REQUESTED	An attempt has started.
2	PENDING with RESPONSE_TIMEOUT	The final result is uncertain.
3	SUCCEEDED	The attempt eventually succeeded; its earlier timeout is not a final failure.

A reason code being present does not automatically make the event a failure. Check both the event status and reason_class, and reconstruct the latest known state before calculating final outcome metrics.

Metadata JSON identity and observation window

dataset_metadata.metadata_json is one JSON object. The loader inserts it under metadata_key = generation_config. These nested properties are settings inside that column, not additional SQL columns.

JSON property	Type and default	Meaning
seed	Integer 42	Seed used for repeatable random generation. Reuse the same code, settings, and Python version for the same output.
start_date	String 2026-07-01	First local business date in the observation period.
days	Integer 90	Length of the simulation's observation window in local days.
customers	Integer 5000	Number of generated customer records.
payments	Integer 100000	Number of logical payment requests; retries create additional attempts.
generator_version	String 1.0.0	Version label of the data generator.
observation_start_utc	String timestamp 2026-06-30T21:00:00.000Z	First included instant, expressed in UTC.
observation_cutoff_exclusive_utc	String timestamp 2026-09-28T21:00:00.000Z	End boundary. Events at or after this instant are excluded.
business_timezone	String Africa/Mogadishu (UTC+03:00)	Business timezone used to interpret local dates.
provider_id	String SIM_WALLET	Fictional provider label for the dataset.
synthetic	Boolean true	Confirms that the records are generated rather than real customer records.
reason_source_policy	String SYSTEM_ONLY	Documents the project's system-only reason policy.

Metadata JSON rules and probabilities

Probabilities range from 0 to 1: 0.01 means 1 percent. They are simulation settings, not observed failure rates or market statistics.

JSON property	Type and default	Meaning
currency	String USD	Currency used throughout the simulation.
fees_cents	Integer 0	No transfer fees are modelled.
transfer_limit_cents	Integer 50000	Simulated per-transfer limit of USD 500.00; not a regulatory or real-provider limit.
max_attempts	Integer 3	Maximum number of attempts for one logical payment.
duplicate_rate	Number 0.01	For an event not dropped, the probability of an extra feed delivery.
late_rate	Number 0.02	Probability of an additional 1 to 36 hour delay for an event that is not dropped.
drop_rate	Number 0.002	Probability a canonical event is omitted from the analytics feed. The underlying payment still has its own outcome.
retry_probability	Number 0.65	Chance of another attempt after a retryable confirmed failure, subject to the attempt cap and cutoff.
timeout_probability	Number 0.025	Baseline timeout probability for an otherwise eligible attempt. Incident handling can increase it.
unresolved_given_timeout	Number 0.04	Chance that a timeout receives no scheduled definitive resolution in the simulation. Cutoff timing can also leave attempts unresolved.
failure_given_resolved_timeout	Number 0.20	Chance that a timeout scheduled to resolve ends in confirmed failure rather than success.

Realised counts vary because these are probabilities. Some assumptions, including city weights, channel mix, and incident severity, remain in the generator code rather than this JSON object.

Using the tables without changing their meaning

Join at the correct grain

Start with payment_requests for payment-level questions and payment_attempts for attempt-level questions. Linking requests directly to every event delivery multiplies rows. Deduplicate events and derive one outcome per attempt before joining that result into reporting models.

Preserve the three clocks

event_at is the business event time. ingested_at is the source feed arrival time. Your future extraction pipeline should add its own extracted_at and batch_id fields rather than overwriting either source timestamp. These future fields are not columns in the supplied source schema.

Use checkpoints for the intended source model

source_row_id can mark progress through this append-only event table. Advance the checkpoint after the destination batch is safely written. Do not assume this cursor alone detects updates or deletes in other tables, or solves concurrent production change capture.

Keep the answer files outside the pipeline

The evaluation folder contains expected outcomes, canonical events, and delivery-defect labels. These are checking tools, not source features. The loader does not import them into MySQL. A missing confirmation may leave the observed result uncertain even when the answer key knows the actual result.

Distinguish schema rules from simulation rules

PK, FK, required fields, unique keys, and explicit CHECK constraints are SQL rules. Some additional properties are enforced by the generator and its tests: two balanced postings per successful attempt, money conservation, and a maximum of three attempts. A database table definition alone does not enforce every cross-table business rule.

Source files for this reference

File	Role
schema_mysql.sql	Exact MySQL tables, columns, types, keys, and constraints.
generate_data.py	Field generation, system reasons, timestamps, state transitions, and simulation settings.
load_mysql.py	Database-generated source_row_id, CSV-to-NULL conversion, UTC loading, and dataset_metadata insertion.
README.md	Project scope, default dataset counts, and source versus evaluation guidance.

This dictionary describes the supplied MySQL source model. Proposed bronze, silver, gold, and mart tables in Databricks or Snowflake are downstream models that you will design separately.